

AndroidDaily: A Verifiable Benchmark for Mobile GUI Agents on Real-World Closed-Source Applications

YiFan Sui^{1,2,*}, Xin Huang^{2,3,*}, Hongbing Li^{1,2}, Fang Xu², Jiahe Lv², Haolong Yan^{1,2}, Yeqing Shen², Litao Liu², Zhimin Fan², Ziyang Meng², Jia Wang², Junbo Qi², Kaijun Tan², Zheng Ge², Xiangyu Zhang², Daxin Jiang², Osamu Yoshie³

¹Beijing University of Posts and Telecommunications ²StepFun ³Waseda University

Abstract

The rapid development of GUI foundation models and mobile GUI agents has spurred numerous evaluation benchmarks, yet most rely on simulated environments or open-source applications, leaving real-world closed-source applications largely unevaluated. The core difficulty is that closed-source applications do not expose internal states, making traditional automatic verification inapplicable. To bridge this gap, we introduce **AndroidDaily**, a large-scale benchmark comprising 350 realistic daily-use tasks across 94 high-frequency Android applications spanning transportation, shopping, local services, entertainment, content creation, social media, and everyday utilities. To enable automatic and verifiable assessment in these opaque environments, we propose Guideline-grounded Reviewer for Automatic Diagnostic Evaluation (**GRADE**), a process-aware evaluator built on a three-tiered system of observable external guidelines: *operational obligations*, *output quality*, and *negative constraints*. GRADE tracks the agent’s visual trajectory against these criteria and produces step-level diagnostic judgments, turning long-horizon, open-ended mobile interactions into verifiable evaluation without relying on hidden internal states. Experiments show that GRADE achieves 87.37% agreement with human evaluators. The strongest model reaches a 62.0% success rate on AndroidDaily, highlighting a substantial gap between current reasoning capabilities and practical execution in realistic mobile workflows.

CCS Concepts

• **Human-centered computing** → **Ubiquitous and mobile computing design and evaluation methods**.

Keywords

Mobile GUI Agent Benchmark, Closed-Source apps, Process-Aware Evaluation

1 Introduction

GUI agents, intelligent systems capable of perceiving, understanding, and autonomously operating graphical user interfaces, hold the promise of transforming how users interact with mobile devices, from automating repetitive tasks to enabling hands-free accessibility [21, 35, 42]. As these systems move closer to real-world deployment, rigorous and systematic evaluation becomes essential for understanding their capabilities and guiding further improvement. Early efforts primarily target static settings, assessing an agent’s ability to perceive and localize interface elements through tasks

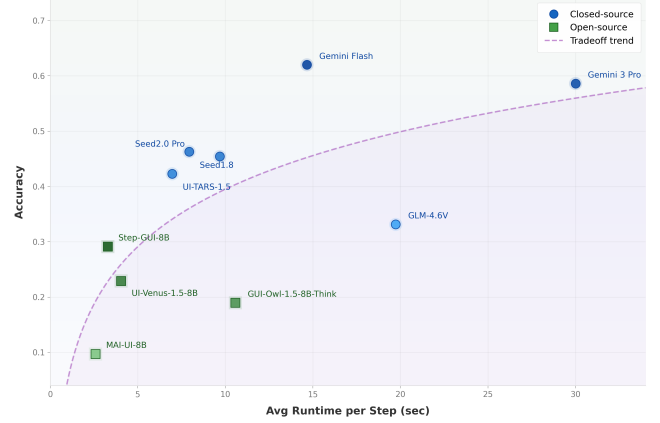


Figure 1: Relationship between per-step inference latency and task success rate on AndroidDaily. Closed-source models generally trade higher latency for better performance, while open-source models cluster at low latency but lower accuracy. Gemini 3 Flash achieves the best accuracy–latency tradeoff.

such as widget captioning, screen summarization, and element grounding [23, 29]. More recently, a line of work has revealed a fundamental gap between static grounding proficiency and dynamic execution capability [31], motivating the development of online benchmarks that evaluate agents through closed-loop interaction with live environments [36, 48]. Despite this rapid progress, existing online benchmarks predominantly center on productivity or utility applications (Table 1). A critical gap remains: the disconnect between evaluated tasks and actual daily usage patterns. Automated evaluation on real-world closed-source applications (the apps that dominate everyday mobile use) remains largely unexplored. Unlike AndroidWorld [36] and OSWorld [48], which evaluate agents in open-source or sandboxed environments where success can be verified by injecting probes into application state, the commercial applications that dominate daily mobile use expose no such interface. Existing approaches therefore either restrict themselves to a handful of utility apps with privileged access, or fall back to coarse end-state matching that misses the long-horizon, multi-constraint behaviors characteristic of realistic usage.

To bridge this gap, we introduce AndroidDaily, a large-scale end-to-end mobile GUI agent benchmark grounded in empirical mobile usage patterns (Figure 2). Based on usage frequency data and download statistics, we curate 350 realistic tasks spanning 94 high-frequency closed-source Android applications across key daily-life

*Both authors contributed equally to this research.
Preprint.

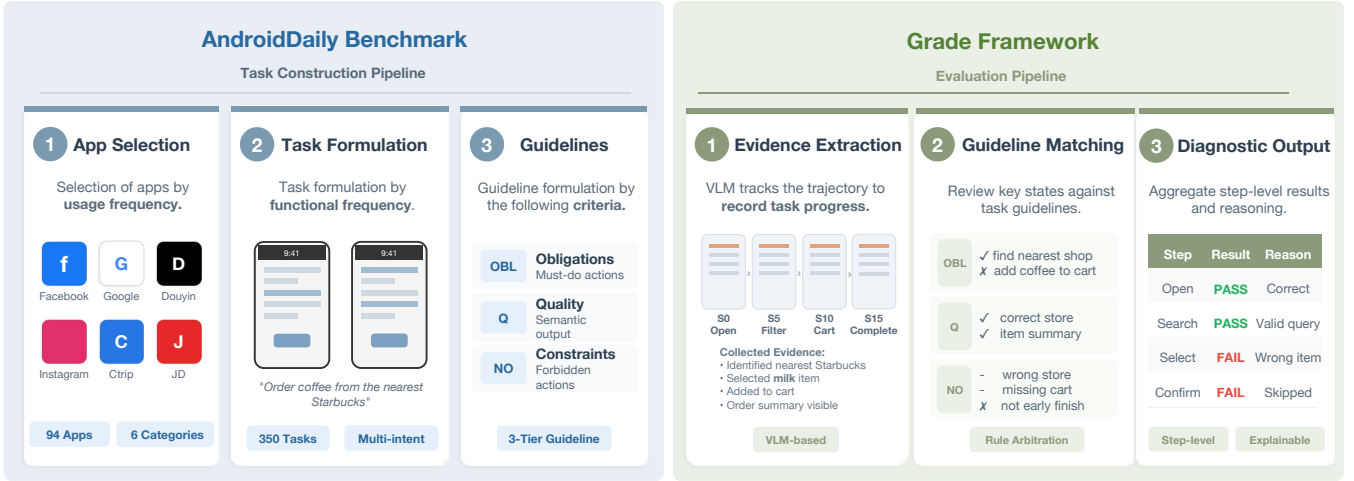


Figure 2: Overview of AndroidDaily and GRADE. Left: the Task Construction Pipeline builds AndroidDaily with 350 realistic tasks across 94 high-frequency apps, each annotated with three-tier guidelines. Right: the GRADE Evaluation Pipeline evaluates agent trajectories against task guidelines and produces step-level diagnostic judgments.

Benchmark	Language	Platform	Tasks	R	On	S
Mind2Web [13]	EN	Web	2350	✓	✗	✗
ScreenSpot [10]	EN	Mobile, Desktop, Web	1272	✓	✗	✗
ScreenSpot Pro [29]	EN	Desktop	1581	✓	✗	✗
ScreenQA [22]	EN	Mobile	85984	✓	✗	✗
MMBench-GUI [43]	EN	Mobile, Desktop, Web	8123	✓	✗	✗
SPA-BENCH [9]	EN+CN	Mobile	340	✗	✓	✗
AndroidWorld [36]	EN	Mobile	116	✗	✓	✗
OSWorld [1]	EN	Desktop	369	✓	✓	✗
GUI-CEval [31]	CN	Mobile	8222	✓	✓	✗
AndroidDaily	EN+CN	Mobile	350	✓	✓	✓

Table 1: Comparison of existing GUI agent benchmarks with our AndroidDaily. R, On, S denote Real-word, Online, and Step-level verification.

scenarios, including transportation, food delivery, shopping, social media, short-video consumption, entertainment, and local services. Unlike utility-focused benchmarks, our tasks involve real-world consequences such as financial transactions and service bookings, multi-step decision-making with competing constraints, and cross-application workflows that require coordinating information across multiple apps. These characteristics make AndroidDaily substantially more representative of the scenarios where agent deployment has immediate practical impact.

Evaluating open-ended tasks on real-world closed-source applications introduces a fundamental challenge: internal application states (backend databases, complete UI trees, API responses) are entirely inaccessible, rendering code-based assertions of the kind employed by AndroidWorld and OSWorld inapplicable. Meanwhile, the tasks themselves admit inherent flexibility: a single goal may have multiple valid execution paths, different acceptable operation orderings, and varying legitimate stopping points. Together,

these factors make deterministic automatic evaluation either infeasible or unreliable. To address this, we propose GRADE (Guideline-grounded Reviewer for Automatic Diagnostic Evaluation), a novel evaluation framework that decouples task verification from internal state access. For each task, we define a three-tiered system of external guidelines: (1) *operational obligations* that specify the necessary operations and constraints required for task completion, (2) *output quality* criteria that evaluate the quality of agent-generated content such as summaries, comparisons, or creative text, and (3) *negative constraints* that define actions strictly to be avoided, establishing safety and stopping boundaries. Acting as an LLM-based process-aware evaluator, GRADE tracks the agent’s visual trajectory, matches execution steps against these guidelines, and produces a step-level diagnostic trace rather than a single binary pass/fail verdict, making it possible to pinpoint where and why an agent deviates from the task objective.

Applying AndroidDaily with GRADE to a range of state-of-the-art GUI agent models reveals significant limitations in current capabilities (Figure 1). GRADE achieves 87.37% agreement with human judgments, validating its reliability as an automatic evaluator for closed-source settings. The strongest current model reaches only 62.0% overall success, with all models degrading sharply on multi-constraint and cross-app tasks. Our failure analysis identifies three dominant bottlenecks: *inference latency* causing UI misalignment, *memory-induced action loops*, and *protocol-induced capability degradation*. These findings suggest that the key challenges for next-generation GUI agents lie not only in improving base capabilities, but in simultaneously achieving fast execution, robust history management, and reliable capability preservation under agentic workflows.

Our contributions are as follows:

- We introduce **AndroidDaily**, a large-scale end-to-end mobile GUI agent benchmark grounded in real daily usage patterns. It comprises 350 tasks spanning 94 high-frequency

closed-source Android applications across diverse everyday scenarios, emphasizing cross-app workflows, multi-constraint decision-making, and feedback-driven interaction.

- We propose **GRADE**, an automated process-aware evaluation framework tailored to closed-source environments. By leveraging a three-tiered system of observable guidelines (operational obligations, output quality, and negative constraints), GRADE enables verifiable and step-level diagnostic assessment without requiring access to internal application states.
- We conduct systematic experiments on AndroidDaily using state-of-the-art GUI agent models. Our results reveal substantial capability gaps under realistic daily-use settings and identify three dominant failure modes: latency-induced misalignment, memory-induced action loops, and protocol-induced capability degradation, establishing a baseline and roadmap for future research.

2 Related Work

2.1 GUI Agents

Recent advances in Multimodal Large Language Models (MLLMs) have driven rapid progress in autonomous GUI agents. Current approaches broadly fall into two paradigms. End-to-end models directly map visual observations and natural-language instructions to executable actions through a single unified model. Early explorations such as CogAgent [21], SeeClick [10], and Auto-UI [57], together with related multimodal models such as Fuyu [5], demonstrated the viability of this approach. More recent work, including UI-TARS [35], Step-GUI [52, 53], Qwen3-VL [4], and Claude Opus 4.6 [3], has further advanced performance. Models such as Ferret-UI [56] and Ferret-UI Lite [54] have improved mobile UI understanding and lightweight deployment, while open foundation action models such as OS-Atlas [46], together with generalized automation models like EvoCUA [51] and GUI-Owl, introduced in Mobile-Agent-v3 [55], further broaden the applicability of this paradigm. In parallel, modular frameworks wrap off-the-shelf MLLMs in tool-use or multi-agent pipelines. On mobile, AppAgent [59], AutoDroid [44], DroidBot [32], and the Mobile-Agent series [42, 55] have advanced API-less interaction and exploration mechanisms, while desktop and web frameworks such as OS-Copilot [45], UFO [58], Agent S [2], Cradle [39], WebVoyager [19], and CoAct [38] have demonstrated increasingly capable cross-application workflows, often aided by pixel-level parsing tools like OmniParser [40]. Despite this progress in both paradigms, evaluating these agents on real-world closed-source commercial applications remains fundamentally difficult, as such apps provide no access to privileged internal APIs or underlying backend states.

2.2 Evaluation of GUI Agents

Static evaluation. Early work assessed GUI agents on static perception tasks: identifying interface elements, inferring semantics from single screenshots, or predicting isolated actions. Foundational datasets like Rico [12] enabled tasks such as Widget Captioning [30] and screen summarization via Screen2Words [41]. For

action-oriented grounding, ScreenSpot [10] established a cross-platform benchmark for UI element localization, later extended by ScreenSpot-Pro [29] to complex professional software, while OS-Atlas [46] revised the original ScreenSpot annotations and released the corrected benchmark as ScreenSpot-V2. OmniACT [23] broadened static evaluation to desktop and web action generation. Large-scale trajectory datasets including AITW [37] and Mind2Web [13] further enriched this line of research. While these benchmarks advanced basic visual perception and grounding, they only weakly capture real-world deployment challenges such as multi-step coordination and error recovery.

Dynamic evaluation. To address these limitations, the field shifted to online, closed-loop evaluation. Building on earlier synthetic environments such as MiniWoB++ [34], later work introduced more realistic execution-based environments for web and desktop interaction, including WebArena [62] and OSWorld [48], followed by VisualWebArena [25], BrowserGym [11], Windows Agent Arena [6], AssistGUI [15], CRAB [50], and MMBench-GUI [43]. On mobile, AndroidWorld [36] offered dynamic evaluation with parameterized tasks, and more recent benchmarks such as MobileWorld [26] and GUI-CEval [31] expanded task difficulty, cross-app interaction, and app coverage. Recent efforts like OSWorld-Human [1] further introduced human-efficiency baselines. Many of these benchmarks verify task completion through code-based assertions that inspect benchmark-maintained state, API responses, or system variables—an approach that is highly effective in open-source or simulated settings but hard to scale robustly to closed-source commercial apps, where backend states are hidden and evaluators must often rely on fragile UI-tree heuristics or superficial screen matching.

LLM-based evaluation. Beyond GUI-specific benchmarks, the broader NLP community has increasingly adopted LLM-based evaluation as a scalable alternative to human annotation. Pioneering work such as MT-Bench and Chatbot Arena [60] demonstrated that strong LLMs can serve as reliable pairwise judges for open-ended text generation. Subsequent efforts including JudgeLM [63], which fine-tuned dedicated judge models to mitigate positional and knowledge biases, and CritiqueLLM [24], which trained models to produce informative diagnostic critiques beyond scalar scores, have advanced the reliability and interpretability of automated evaluation. Comprehensive surveys [18, 28] have further systematized this rapidly growing paradigm. In a related direction, process reward models [33] have demonstrated that step-level supervision can yield more faithful credit assignment than outcome-only scoring in mathematical reasoning. Recent work has extended this idea to agent settings, with AgentPRM [47] introducing promise-progress decomposition for sequential decision-making, and ToolPRMBench [27] providing a dedicated benchmark for evaluating process-level rewards in tool-use scenarios. However, these ideas remain much less developed in GUI-agent evaluation than in text-only reasoning or general tool-use settings, especially for long-horizon trajectories on visually complex, closed-source commercial interfaces. Existing GUI benchmarks either rely on code-based assertions tied to privileged state access, or fall back to coarse binary judgments. GRADE bridges this gap by combining the LLM-as-a-Judge paradigm with process-aware trajectory tracking and externally defined three-tiered guidelines, enabling verifiable and diagnosable evaluation on

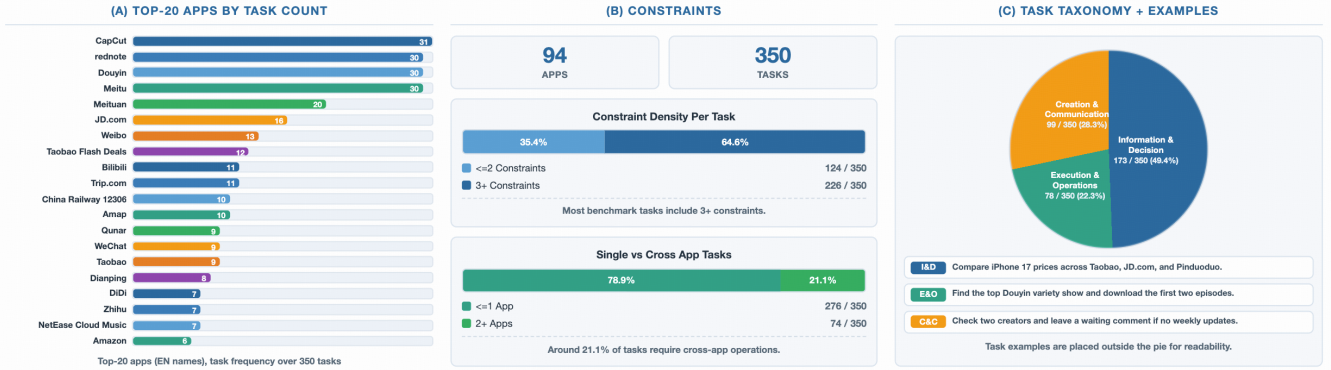


Figure 3: Benchmark statistics of AndroidDaily. (a) Top-20 apps by task frequency (English app names) over 350 tasks, led by CapCut (31), rednote (30), Douyin (30), and Meitu (30). (b) Benchmark scale and constraint density: 94 apps, 350 tasks, with ≤ 2 constraints (124, 35.4%) vs 3+ constraints (226, 64.6%). (c) Task taxonomy pie with concise examples: Information & Decision (173, 49.4%), Execution & Operations (78, 22.3%), and Creation & Communication (99, 28.3%).

closed-source commercial applications without requiring internal state access.

3 Method

This section presents the design of AndroidDaily in detail. We begin with an overview of the benchmark scope and task construction principles (§3.1), describe the guideline-centric task specification that defines externally checkable completion criteria (§3.2), and finally introduce GRADE, the process-aware evaluation framework that operationalizes these guidelines into automatic diagnostic assessment (§3.3).

3.1 AndroidDaily Benchmark

The core design principle of AndroidDaily is to closely mirror real-world mobile usage dynamics. We pursue this goal through two strategies: empirical application selection and diverse task formulation.

Application Selection. Rather than sampling from broad catalogs or relying on open-source alternatives, we ground our selection in empirical usage data to retain high-frequency closed-source applications that cover major daily-use categories. Based on app store download rankings and publicly available active-user statistics, we curate a set of 94 closed-source applications. These apps span the categories users most routinely engage with, including transportation, food delivery, e-commerce, social media, short-video platforms, entertainment, and local lifestyle services.

Task Formulation. For each app, we design tasks that reflect its core functions, common usage patterns, and recurring user intents. Instructions are formulated in natural language with varying levels of specificity to mimic real user behavior, ranging from *precise directives* (e.g., “order a luxury car from Location A to Location B”) to *underspecified intents* (e.g., “play something funny”), which require the agent to infer user preferences and navigate ambiguity. **Task Allocation and Scale.** Dynamic GUI benchmarks typically contain a limited number of tasks due to the high cost of real-device

execution (for instance, OSWorld includes 369 tasks and AndroidWorld 116 tasks). To achieve sufficient breadth while keeping total evaluation time manageable, we adopt a frequency-weighted allocation strategy: apps with higher daily usage frequency are assigned more tasks covering a wider range of functionalities, while lower-frequency apps are tested only on their most essential use cases. This ensures that evaluation effort is concentrated where it matters most. Figure 3 illustrates the task distribution across the top-20 apps in AndroidDaily. The resulting benchmark comprises 350 tasks across 94 applications. Tasks are grouped into three capability-oriented categories:

- **Information & Decision:** Retrieving, comparing, and synthesizing information to support user decisions, ranging from short-horizon lookups to cross-app exploration.
- **Creation & Communication:** Generating user-visible content or engaging in semantic interactions such as composing posts, editing multimedia, commenting, and messaging.
- **Execution & Operations:** Completing concrete actions such as purchasing, booking, navigating, or adjusting settings, with an emphasis on step-by-step reliability and strict constraint satisfaction.

Together, these categories ensure that AndroidDaily is comprehensive in the range of capabilities it exercises: visual perception, intent understanding, long-horizon planning, cross-app coordination, content generation, and precise manipulation.

3.2 Guideline-Centric Task Specification

While AndroidDaily reflects realistic mobile usage, it also brings a practical challenge: most task instructions are open-ended and underspecified. Instead of providing explicit step-by-step commands, users may ask for things like “Any recently trending TV shows? I want something to binge-watch.” or “Help me find a place to take the kids on the weekend.”, leaving the agent to infer missing constraints before acting. Evaluating task completion from the instruction alone is therefore inherently ambiguous. To address this, we pair each

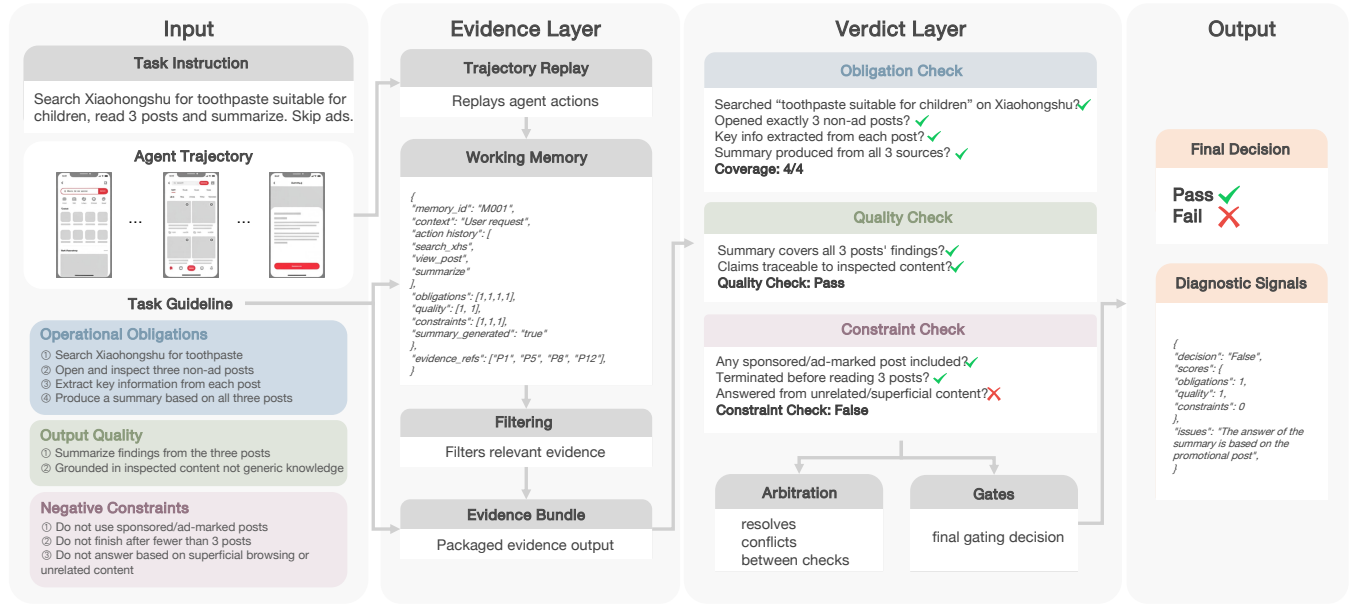


Figure 4: The GRADE evaluation pipeline. The Evidence Layer replays the agent trajectory to assemble structured evidence. The Verdict Layer checks obligations, quality, and boundaries to produce a pass/fail decision with diagnostic signals.

task with an external guideline that converts the open-ended instruction into explicit, observable completion conditions through three complementary tiers.

Operational Obligations. This tier specifies the core operations and intermediate states that must be satisfied for a task to be considered complete, regardless of the specific execution path taken. It encompasses target objects, key constraints, and necessary operations, providing the primary reference for judging basic task fulfillment.

Output Quality. This tier defines criteria for evaluating the quality of the agent’s final state or generated content. Many daily-use tasks require the agent to produce textual outputs (e.g., summaries, recommendations) or specific visual outcomes. These criteria ensure that evaluation captures not just whether the agent performed the right actions, but whether its deliverables meet the task’s semantic and informational goals.

Negative Constraints. This tier defines behaviors that must strictly be avoided, including boundary violations, invalid completion patterns, or safety hazards. A trajectory may appear successful on the surface yet fail if it violates a key constraint (for instance, executing a final payment that the user only asked to preview). Negative constraints prevent the evaluation from being misled by superficially plausible end states.

Together, these three tiers act as a standardized evaluation contract. By translating open-ended user intents into explicit, observable constraints, this specification effectively eliminates subjective ambiguity, ensuring high consistency and reliability in the subsequent automated evaluation.

3.3 GRADE Evaluator

Given the guideline, the remaining question is how to apply it to an agent’s execution trajectory without access to internal application states. In realistic closed-source tasks, the terminal screen alone is insufficient: many decisive cues appear only briefly in intermediate steps and may later be overwritten. GRADE addresses this by operating as a **Vision-Language Model (VLM)-based** evaluator that takes the full multimodal trajectory (a sequence of screenshots and available accessibility metadata) alongside the external guideline (Figure 4). It organizes the evaluation into two layers:

Evidence Layer. This layer tracks, filters, and compresses the visual trajectory into structured evidence. Concretely, GRADE maintains a working memory throughout the trajectory replay, visually extracting and recording task-relevant signals: action history, confirmed facts, target bindings, comparison states, and blocker signals. Uninformative steps are filtered out, yielding a structured *evidence bundle*. This design ensures that transient but critical visual cues (such as a price comparison visible only on an intermediate screen) are retained for judgment.

Verdict Layer. This layer performs a structured review of the evidence bundle against the three-tiered guideline. *Operational obligations* are checked for coverage; *output quality* is checked for semantic adequacy; and *negative constraints* are checked for boundary violations. Beyond item-level review, the verdict layer handles conflicting evidence through explicit arbitration rules (for instance, an unresolved blocker can enforce a failure decision even if other obligations appear satisfied).

Algorithm 1 specifies the complete two-layer pipeline. The evidence layer walks through the trajectory step by step, updating a working memory M and accumulating step-level evidence P , then compiles a structured evidence bundle B . The verdict layer checks

Algorithm 1 GRADE: Two-Layer Evaluation Procedure

Require: Task instruction x ; Guideline $G = \{g^{obl}, g^{qual}, g^{neg}\}$; Trajectory $\tau = \{(o_1, a_1), \dots, (o_T, a_T)\}$

Ensure: Final verdict $y \in \{\text{True}, \text{False}\}$; Diagnostic signals D

- 1: $M \leftarrow \emptyset$; $P \leftarrow \emptyset$
- Evidence Layer**
- 2: **for** each $(o_t, a_t) \in \tau$ **do**
- 3: $p_t \leftarrow \text{TRACEEVIDENCE}(x, G, o_t, a_t)$
- 4: $M \leftarrow \text{UPDATEMEMORY}(M, p_t)$
- 5: $P \leftarrow P \cup \{p_t\}$
- 6: **end for**
- 7: $B \leftarrow \text{BUILDEVIDENCEBUNDLE}(x, G, \tau, M, P)$
- Verdict Layer**
- 8: $v^{obl} \leftarrow \text{CHECKOBLIGATIONS}(g^{obl}, B)$
- 9: $v^{qual} \leftarrow \text{CHECKOUTPUTQUALITY}(g^{qual}, B)$
- 10: $v^{neg} \leftarrow \text{CHECKNEGATIVECONSTRAINTS}(g^{neg}, B)$
- 11: $y \leftarrow \text{ARBITRATE}(v^{obl}, v^{qual}, v^{neg}, B)$
- 12: $D \leftarrow \text{BUILDDIAGNOSTICS}(v^{obl}, v^{qual}, v^{neg}, B)$
- 13: **return** (y, D)

B against each of the three guideline tiers and an arbiter combines the slot verdicts into the final decision y together with diagnostic signals D . This design makes GRADE’s reliability depend on two distinct capabilities: fine-grained visual recognition at the evidence layer, and structured reasoning over aggregated evidence at the verdict layer.

GRADE ultimately outputs a final completion decision alongside a set of diagnostic signals (e.g., obligation coverage, boundary violations, and failure-reason tags). This structured output makes decisions more stable than one-shot LLM judgments and supports downstream error analysis.

An Open Evaluation Protocol. Taken together, the task set, the guideline specification, and the GRADE evaluator form a modular and extensible paradigm. Because GRADE is task-agnostic and operates solely on the instruction–guideline–trajectory triplet, these components are not tightly coupled. Users can define new tasks on arbitrary apps, write custom guidelines, and run GRADE without any framework modification. AndroidDaily is therefore not just a fixed dataset, but an open evaluation protocol: flexible in task scope, customizable in evaluation standards, and applicable to any realistic mobile scenario.

4 Experiments

We evaluate multiple state-of-the-art GUI agent models on AndroidDaily, validate the reliability of GRADE against human annotators, and analyze the distinctive failure modes that emerge from large-scale evaluation.

4.1 Experimental Setup

Models. We evaluate twelve mobile GUI agent models organized by training paradigm: *general-purpose VLMs* adapted for GUI interaction and *GUI-specialized models* trained with dedicated agent pipelines. The general-purpose group includes Gemini 3 Flash and Gemini 3 Pro [17], Seed1.8 [7] and Seed2.0 Pro [8], and GLM-4.6V [20]. The GUI-specialized group includes UI-TARS-1.5 [35]

(a closed-source system), Step-GUI-8B and Step-GUI-30A3 [53], UI-Venus-1.5-8B [14], GUI-Owl-1.5-8B-Think and GUI-Owl-1.5-32B-Think [49], and MAI-UI-8B [61]. This taxonomy is orthogonal to the open-/closed-source distinction and instead highlights the role of training paradigm: whether strong GUI performance emerges from general capabilities or requires domain-specific supervision.

Rollout protocol. For each model, we adopt its officially released inference protocol when available; for models without a public protocol, we tune the configuration to achieve the best performance before evaluation. Following the implementation of StepGUI [53], all agent actions are executed on real Android devices via ADB. Task-specific environment preconditions (e.g., logged-in accounts, pre-loaded content, or required app states) are prepared before each session. During execution, a human monitor is present to intervene only in cases of unexpected device states such as screen-off or system-level interruptions, without influencing the agent’s decisions. Each session is capped at a maximum of 60 interaction steps and 40 minutes of wall-clock time, and evaluated under the pass@1 setting. The full execution process is recorded as a chain-of-action trajectory indexed by a unique session ID, which serves as the input to GRADE for subsequent evaluation.

4.2 Main Benchmark Results

We report the overall benchmark results of representative mobile GUI agents on AndroidDaily. Table 2 summarizes success rates across the full benchmark together with per-step latency and three objective slicing dimensions: guideline constraint count, app scope, and task taxonomy. For proprietary models, latency reflects API round-trip time; for locally deployed models, it reflects BF16 inference on H20 GPUs.

Overall results. AndroidDaily clearly separates model tiers. The top-performing systems are all general-purpose frontier VLMs: Gemini 3 Flash achieves the highest overall success rate of 62.0%, followed by Gemini 3 Pro (58.6%), Seed1.8 (46.3%), and Seed2.0 Pro (45.4%). GUI-specialized models lag substantially behind: the strongest is UI-TARS-1.5 at 42.3%—a large-scale closed-source system trained with dedicated GUI agent pipelines including reinforcement learning—yet it still falls short of every general-purpose frontier model. The remaining GUI-specialized models drop further to 29.7% (Step-GUI-30A3) and below. A closer examination of trajectory logs reveals a key qualitative difference: frontier models exhibit markedly stronger *error-path recovery*. In real-device execution, agents routinely encounter environmental interference—slow page loads, unexpected pop-ups, landing on unintended screens, or dynamic content shifts. Frontier models recover from these disruptions more reliably by re-planning or backtracking, whereas GUI-specialized models tend to continue executing on the derailed path or enter repetitive loops. This robustness to real-world noise, rather than GUI grounding accuracy alone, accounts for much of the performance gap.

Latency and reasoning depth. As visualized in Figure 1, latency interacts with success rate differently across the two model groups. Among general-purpose frontier models, longer per-step inference time generally correlates with higher success rates: Gemini 3 Flash (14.7s/step, 62.0%) outperforms Seed1.8 (7.9s/step, 46.3%) and Seed2.0 Pro (9.7s/step, 45.4%). This suggests that the additional

Table 2: Main benchmark results on AndroidDaily. Task success rates (%) are reported by constraint count, app scope, and task taxonomy. Runtime is the average seconds per step over valid sessions. Models are grouped by training paradigm: general-purpose VLMs vs. models trained with dedicated GUI agent pipelines. Best results within each group are bolded.

Model	Avg Step Time (s)	Overall	Constraints		App Scope		Task Taxonomy		
			≤2	3+	≤1 App	2+ Apps	Info.&Dec.	Crea.&Com.	Exec.&Ops.
General-purpose VLMs									
Gemini 3 Flash [16]	14.7	62.0	69.4	58.0	62.7	59.5	63.6	61.6	59.0
Gemini 3 Pro [17]	30.0	58.6	74.2	50.0	63.0	41.9	60.7	50.5	64.1
Seed1.8 [7]	7.9	46.3	58.9	39.4	48.9	36.5	47.4	46.5	43.6
Seed2.0 Pro [8]	9.7	45.4	55.6	39.8	46.0	43.2	46.8	44.4	43.6
GLM-4.6V [20]	19.7	33.1	43.5	27.4	35.9	23.0	36.4	23.2	38.5
GUI-specialized models									
UI-TARS-1.5 [35]	7.0	42.3	54.0	35.8	47.1	24.3	41.6	43.4	42.3
Step-GUI-30A3 [53]	2.0	29.7	41.9	23.0	33.0	17.6	31.8	23.2	33.3
Step-GUI-8B [53]	3.3	29.1	42.7	21.7	33.3	13.5	32.9	23.2	28.2
UI-Venus-1.5-8B [14]	4.0	22.9	31.5	18.2	26.8	8.2	24.4	18.2	25.6
GUI-Owl-1.5-32B-Think [49]	25.6	21.6	33.6	15.1	25.5	7.4	29.7	9.3	20.3
GUI-Owl-1.5-8B-Think [49]	10.6	19.0	29.5	13.3	22.3	6.8	23.7	10.3	19.2
MAI-UI-8B [61]	2.6	9.7	15.3	6.6	12.0	1.4	8.7	13.1	7.7

Table 3: Layer ablation of GRADE on 879 manually reviewed sessions. Adding the verdict layer primarily reduces false positives.

Configuration	N	Acc.(%)	TP	TN	FP	FN
Evidence Layer only	879	84.76	185	560	106	28
+ Verdict Layer	879	87.37	188	580	86	25

Table 4: Pairwise agreement (%) on the 100-session challenging subset. Each cell reports with / without guideline. Only the lower triangle is shown.

	GT	GRADE	H1	H2	H3	H4	H5
GRADE	73 / 72						
H1	63 / 60	56 / 62					
H2	67 / 54	64 / 56	68 / 68				
H3	61 / 57	56 / 67	64 / 71	70 / 77			
H4	56 / 50	59 / 56	69 / 68	63 / 74	69 / 73		
H5	63 / 49	60 / 63	72 / 71	74 / 81	76 / 82	79 / 79	

computation is spent on deeper reasoning—constraint tracking, cross-app coordination, and recovery planning—which directly benefits complex daily-use tasks. The correlation breaks down at the extreme: Gemini 3 Pro spends 30.0s per step yet scores lower than Flash (58.6% vs. 62.0%), because its high per-step latency accumulates on long-horizon tasks, frequently pushing sessions past the 40-minute time limit.

Among GUI-specialized models, however, no such latency-accuracy correlation is observed. UI-TARS-1.5, the strongest in this group, achieves 42.3% at just 7.0s/step, yet GUI-Owl-32B is more than three times slower (25.6s/step) while scoring only 21.6%. Within model families, scaling yields negligible gains: Step-GUI improves by just 0.6pp from 8B to 30A3, and GUI-Owl by 2.6pp from 8B to 32B. The additional parameters or computation do not translate into

Table 5: Backbone comparison for GRADE on 879 manually reviewed sessions. Only the backbone model inside GRADE is changed across rows; the protocol, guidelines, and decision logic are held fixed.

Evaluator Backbone	N	Acc.(%)	TP	TN	FP	FN
Gemini 3 Pro	879	87.37	188	580	86	25
Gemini 3 Flash	879	80.55	193	515	151	20
Seed1.8	879	83.73	175	561	105	38
Seed2.0 Pro	879	84.19	195	545	121	18
GPT-4o	879	65.64	190	387	279	23

meaningfully better task completion. This contrast suggests that the reasoning patterns underlying frontier models’ success—error recovery, constraint satisfaction under ambiguity, long-horizon re-planning—are not well captured by current GUI-specialized training pipelines. Closing this gap likely requires richer training data that emphasizes these capabilities, rather than parameter scaling alone. **Slice-level analysis.** Even frontier models are far from solving AndroidDaily. Performance degrades consistently as task complexity increases. On the constraint dimension, every model drops from ≤ 2 to ≥ 3 constraints; for example, Gemini 3 Flash falls from 69.4% to 58.0%, and Seed1.8 from 58.9% to 39.4%. The app scope dimension reveals an even sharper decline: Gemini 3 Pro drops from 63.0% to 41.9% on multi-app tasks. Across the task taxonomy, Creation & Communication tasks are generally harder than Execution & Operations, particularly for GUI-specialized models (Step-GUI-8B: 23.2% vs. 28.2%), likely because they require generating contextually appropriate content beyond GUI manipulation. Overall, AndroidDaily not only separates models by aggregate success rate, but also systematically exposes weaknesses as tasks become more compositional and realistic—even for the strongest frontier systems.

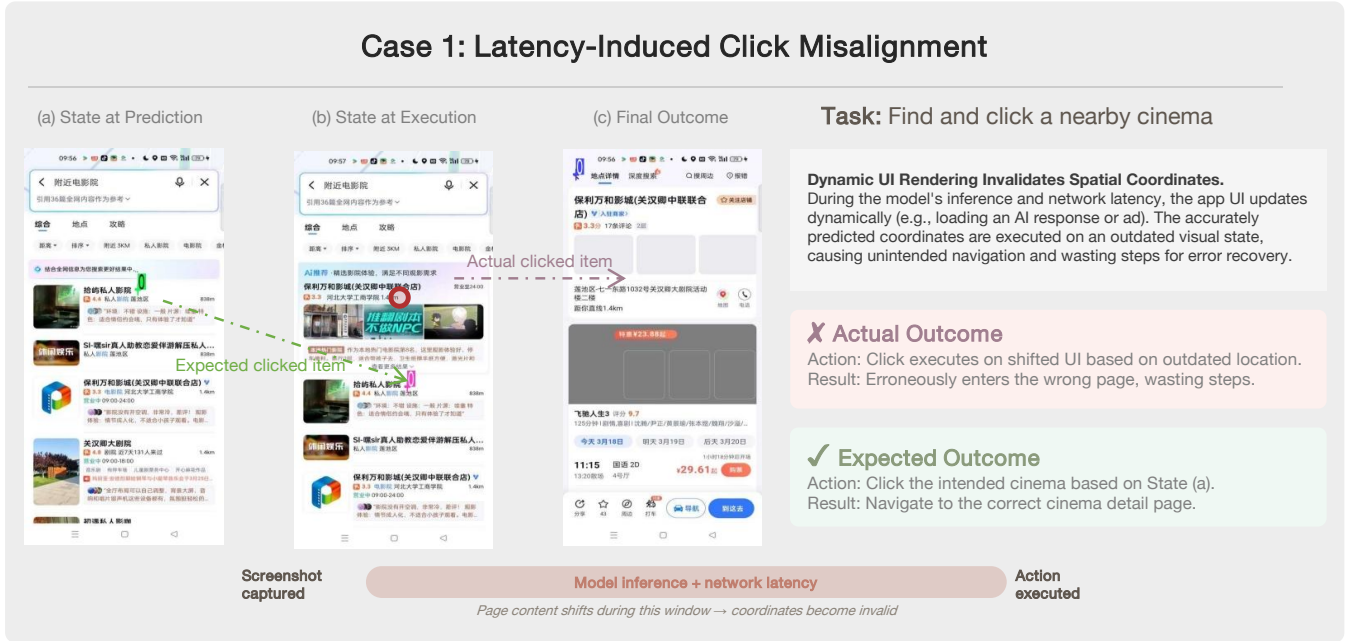


Figure 5: Illustration of Latency-Induced Click Misalignment. (a) At the prediction state, the model accurately plans spatial coordinates (green box) to click the target cinema. (b) During the execution phase, model inference and network latency allow the dynamic UI to update (e.g., inserting an AI response), shifting the intended target downward. The agent executes the click at the outdated coordinates, mistakenly hitting a different item (red circle). (c) The final outcome shows the agent navigating to an unintended page, which derails the task and wastes subsequent recovery steps.

4.3 Evaluator Reliability

To assess GRADE’s evaluation quality, we collected 879 sessions from the raw inference outputs across all baseline models. Each session was manually reviewed and calibrated through three successive rounds of human annotation to produce a reliable ground-truth label. This adjudicated label set serves as the reference for all subsequent reliability analyses.

Layer ablation. We first ablate GRADE’s two-layer design. As shown in Table 3, the evidence layer alone already achieves 84.76% accuracy (TP=185, TN=560, FP=106, FN=28). Adding the verdict layer primarily suppresses false positives from 106 to 86, while slightly improving true positives (188) and true negatives (580). The full two-layer pipeline reaches 87.37% accuracy, confirming that both layers contribute meaningfully: the evidence layer provides a robust baseline by extracting structured trajectory evidence, while the verdict layer refines it by performing fine-grained constraint checking that catches trajectories that superficially appear successful but miss critical details.

Backbone comparison. To verify that GRADE’s reliability stems from the protocol rather than the choice of a single backbone, we swap five VLMs into the same GRADE procedure (Table 5). The backbone’s visual perception capability has a decisive impact on verification accuracy. While true-positive counts remain stable across all backbones (175–195), false-positive counts vary dramatically—from 86 (Gemini 3 Pro) to 279 (GPT-4o). The vast majority of these false positives trace back to the same root cause: the backbone fails to detect a subtle error in the trajectory (e.g., a wrong item selected,

a constraint silently unmet) and propagates this misperception into an incorrect positive verdict.

Human agreement on hard cases. This pattern is not unique to models. We selected 100 visually ambiguous trajectories and recruited 5 non-expert evaluators to independently judge their success. Table 4 records pairwise agreement among all raters, including GRADE, the calibrated expert ground truth (GT), and the five annotators, under both with- and without-guideline settings. The results show that human evaluators struggle with the same root cause as model backbones: fine-grained visual details that determine task success are easy to overlook. Without guidelines, individual human–GT agreement ranges from just 49% to 60%; with guidelines it improves to 56–67%, primarily by reducing false positives—annotators become less likely to accept superficially plausible trajectories that actually miss key constraints. Across both settings, GRADE maintains 72–73% agreement with GT, consistently matching or exceeding every individual non-expert annotator. The difference is that models occasionally *cannot perceive* critical visual cues, while humans *do not attend to them carefully enough*—but the practical effect on evaluation reliability is similar. GRADE thus serves as a consistent and scalable automatic proxy for large-scale evaluation on AndroidDaily.

4.4 Failure Mode Analysis

We summarize the most common failure patterns observed on AndroidDaily. To ground this analysis quantitatively, we manually categorized all failed sessions from the top three frontier and top

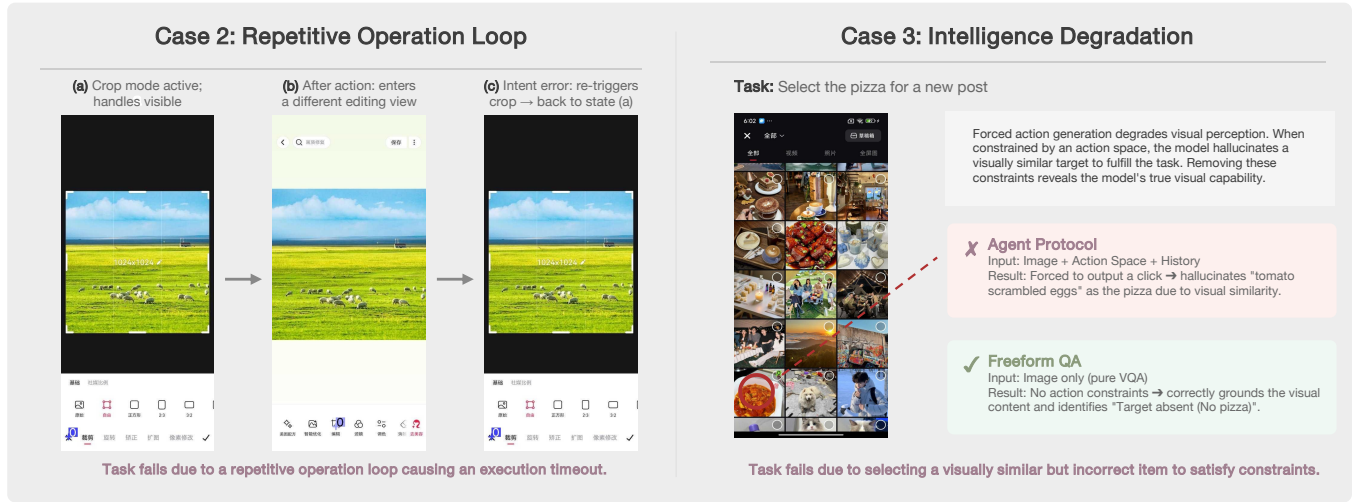


Figure 6: Two more failure cases under realistic mobile evaluation. (Left) Repetitive Operation Loop: The agent becomes trapped in a cyclic execution path. After transitioning from the initial state (a) to a new view (b), an intent error causes it to revert to the same initial state (c), creating an infinite loop that results in an execution timeout. **(Right) Intelligence Degradation:** Augmenting the input with agent-specific context (action space definitions, interaction history, etc.) can probabilistically degrade the model’s visual perception. When tasked to select “pizza” (absent from the screen), a pure VQA query correctly identifies its absence, whereas the same model under the agent protocol hallucinates a click on a visually similar item (“tomato scrambled eggs”), causing task failure.

three GUI-specialized models, identifying four dominant failure types.

Frontier models: latency as the dominant bottleneck. Despite stronger reasoning and recovery capabilities, frontier models are primarily limited by high inference latency, which manifests in two ways. First, *timeout failures*: on long-horizon tasks (≥ 3 constraints or ≥ 3 apps), accumulated per-step latency frequently pushes sessions past the 40-minute wall-clock limit, accounting for the sharp performance drop observed in Table 2 for Gemini 3 Pro on these slices. Second, *action misalignment with dynamic interfaces*: as shown in Figure 5, the agent decides to click a target element, but during the delay between decision and execution, a transient UI element (e.g., an AI recommendation overlay, a pop-up, or a loading screen) appears and shifts the intended click target. The tap lands on the wrong element, forcing the agent to dismiss it and navigate back, wasting additional steps and compounding the latency problem.

GUI-specialized models: decision quality as the primary weakness. GUI-specialized models are faster but make substantially more decision errors, with two patterns standing out. First, *memory-induced action loops*: as shown in Figure 6 (left), the agent loses track of its history and cycles between a few states indefinitely, exhausting the step budget without progress. Second, *protocol-induced capability degradation*: in Figure 6 (right), the agent misidentifies scrambled eggs with tomatoes as pizza during GUI execution, yet correctly refuses to click “pizza” when the same image is presented in a free-form VQA setting. The perceptual capability exists but fails to activate under the structured action protocol, suggesting that agent-specific context (action space definitions, interaction

history, structured output format) can probabilistically interfere with the underlying VLM’s visual reasoning.

Implications for future development. These failure patterns point to three concrete research directions: (i) *latency-aware agent design*, including speculative action planning, asynchronous execution, or lightweight verification modules that can handle dynamic UI changes without full model re-inference; (ii) *robust long-horizon memory*, such as explicit state tracking or trajectory summarization mechanisms that prevent action loops; and (iii) *capability-preserving agentic protocols* that maintain the base VLM’s perceptual accuracy when augmented with agent-specific context. The contrasting failure profiles of frontier and GUI-specialized models further suggest a fourth direction: (iv) *training data that captures recovery and re-planning patterns*, which frontier models appear to acquire from general pretraining but GUI-specialized pipelines currently lack. AndroidDaily, by requiring reasoning depth, fast response, and robust capability preservation simultaneously, provides a diagnostic testbed for progress along all four axes.

5 Limitations and Ethics

Evaluation constraints. Following established practice in real-device GUI agent benchmarks [36, 48], we report *pass@1* success rates without multi-seed variance, as each rollout requires up to 40 minutes of wall-clock time on physical Android hardware. Multi-seed variance estimation on a representative subset is a valuable direction for future work. In addition, the commercial applications underlying AndroidDaily evolve continuously through UI updates, A/B testing, and personalized content. We mitigate environmental drift through controlled APK versioning and standardized account states (§4.1), but absolute success rates should be interpreted as a

snapshot at evaluation time; relative comparisons across models evaluated in the same window remain meaningful.

Ethics statement. All rollouts are executed on dedicated research-purpose accounts owned by the authors; no end-user accounts or third-party data are involved. The *negative constraints* tier of our guideline specification (§3.2) is designed in part to prevent agents from completing irreversible real-world actions such as final payments or messages to unrelated parties, and all sessions are monitored by a human operator who can abort boundary-approaching runs. Screenshots in the paper have been reviewed and personally identifiable information has been redacted. We release task definitions, guidelines, and evaluation code rather than raw trajectory data, and encourage downstream users to comply with the applicable terms of service when reusing the protocol on new applications. Extending coverage to professional, enterprise, and accessibility-specific workflows remains a natural direction for future work.

6 Conclusion

We presented AndroidDaily, a large-scale benchmark of 350 realistic daily-use tasks across 94 closed-source Android applications, together with GRADE, a process-aware evaluator that enables verifiable and diagnosable assessment without access to internal application states. GRADE achieves 87.37% agreement with calibrated human labels. The strongest current model reaches only 62.0% success, with all models degrading sharply on multi-constraint and cross-app tasks. Our analysis identifies three dominant failure modes: latency-induced misalignment, memory-induced loops, and protocol-induced capability degradation, pointing to the need for agents that combine reasoning depth, fast execution, and robust capability preservation under agentic protocols. AndroidDaily is designed as an open evaluation protocol and can be freely extended with custom tasks and guidelines.

References

- [1] Reyna Abhyankar, Qi Qi, and Yiyang Zhang. 2025. Osworld-human: Benchmarking the efficiency of computer-use agents. *arXiv preprint arXiv:2506.16042* (2025).
- [2] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. 2024. Agent s: An open agentic framework that uses computers like a human. *arXiv preprint arXiv:2410.08164* (2024).
- [3] Anthropic. 2026. *System Card: Claude Opus 4.6*. Technical Report. Anthropic. <https://www-cdn.anthropic.com/6a5fa276ac68b9aeb0c8b6af5fa36326e0e166dd.pdf>
- [4] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. 2025. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631* (2025).
- [5] Rohan Bavishi, Erich Elsen, Curtis Hawthorne, Maxwell Nye, Augustus Odena, Arushi Somani, and Saġnak Taşlılar. 2023. Introducing our Multimodal Models. <https://www.adept.ai/blog/fuyu-8b>
- [6] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, et al. 2024. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264* (2024).
- [7] ByteDance Seed. 2026. Seed1.8 Model Card: Towards Generalized Real-World Agency. *arXiv preprint arXiv:2603.20633* (2026).
- [8] ByteDance Seed. 2026. *Seed2.0 Model Card: Towards Intelligence Frontier for Real-World Complexity*. Technical Report. ByteDance. <https://lf3-static.bytednsdoc.com/obj/eden-cn/lapzild-tss/jhwZthlaukjlkulzlp/seed2/0214/Seed2.0%20Model%20Card.pdf>
- [9] Jingxuan Chen, Derek Yuen, Bin Xie, Yuhao Yang, Gongwei Chen, Zhihao Wu, Li Yixing, Xurui Zhou, Weiwen Liu, Shuai Wang, et al. 2024. Spa-bench: A comprehensive benchmark for smartphone agent evaluation. In *NeurIPS 2024 Workshop on Open-World Agents*.
- [10] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. 2024. Seedclick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9313–9332.
- [11] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayegan, Lawrence Keunho Jang, Xing Han Lü, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. 2025. The BrowserGym Ecosystem for Web Agent Research. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=5298fKGmv3> Expert Certification.
- [12] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschan, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. 2017. Rico: A mobile app dataset for building data-driven design applications. In *Proceedings of the 30th annual ACM symposium on user interface software and technology*. 845–854.
- [13] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems* 36 (2023), 28091–28114.
- [14] Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, Beitong Zhou, et al. 2026. UI-Venus-1.5 Technical Report. *arXiv preprint arXiv:2602.09082* (2026).
- [15] Difei Gao, Lei Ji, Zechen Bai, Mingyu Ouyang, Peiran Li, Dongxing Mao, Qincheng Wu, Weichen Zhang, Peiyi Wang, Xiangwu Guo, et al. 2023. Assist-gui: Task-oriented desktop graphical user interface automation. *arXiv preprint arXiv:2312.13108* (2023).
- [16] Google DeepMind. 2025. *Gemini 3 Flash*. Technical Report. Google DeepMind. <https://deepmind.google/models/gemini/flash/>
- [17] Google DeepMind. 2025. *Gemini 3 Pro Model Card*. Technical Report. Google DeepMind. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf>
- [18] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on llm-as-a-judge. *The Innovation* (2024).
- [19] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6864–6890.
- [20] GLM-V Team: Wenyi Hong, Wenmeng Yu, Xiaotao Gu, et al. 2025. GLM-4.5V and GLM-4.1V-Thinking: Towards Versatile Multimodal Reasoning with Scalable Reinforcement Learning. *arXiv preprint arXiv:2507.01006* (2025).
- [21] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2024. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 14281–14290.
- [22] Yu-Chung Hsiao, Fedir Zubach, Gilles Baechler, Srinivas Sunkara, Victor Cărbune, Jason Lin, Maria Wang, Yun Zhu, and Jindong Chen. 2025. Screenqa: Large-scale question-answer pairs over mobile app screenshots. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 9427–9452.
- [23] Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem AlShikh, and Ruslan Salakhutdinov. 2024. Omniact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In *European Conference on Computer Vision*. Springer, 161–178.
- [24] Pei Ke, Bosi Wen, Andrew Feng, Xiao Liu, Xuanyu Lei, Jiale Cheng, Shengyuan Wang, Aohan Zeng, Yuxiao Dong, Hongning Wang, et al. 2024. CritiqueLLM: Towards an informative critique generation model for evaluation of large language model generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13034–13054.
- [25] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 881–905.
- [26] Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, et al. 2025. MobileWorld: Benchmarking Autonomous Mobile Agents in Agent-User Interactive and MCP-Augmented Environments. *arXiv preprint arXiv:2512.19432* (2025).
- [27] Dawei Li, Yuguang Yao, Zhen Tan, Huan Liu, and Ruocheng Guo. 2026. Tool-PRMBench: Evaluating and Advancing Process Reward Models for Tool-using Agents. *arXiv preprint arXiv:2601.12294* (2026).
- [28] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. Llm-as-judges: a comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579* (2024).
- [29] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. 2025. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 8778–8786.
- [30] Yang Li, Gang Li, Luheng He, Jingjie Zheng, Hong Li, and Zhiwei Guan. 2020. Widget captioning: Generating natural language description for mobile user

- interface elements. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, 5495–5510.
- [31] Yang Li, Yuchen Liu, Haoyu Lu, Zhiqiang Xia, Hongzhen Wang, Kaiyang Han, Changpeng Yang, Jinyang Wu, Jiaming Xu, Runyu Shi, et al. 2026. GUI-CEval: A Hierarchical and Comprehensive Chinese Benchmark for Mobile GUI Agents. *arXiv preprint arXiv:2603.15039* (2026).
 - [32] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. 2017. Droidbot: a lightweight ui-guided test input generator for android. In *2017 IEEE/ACM 39th international conference on software engineering companion (ICSE-C)*. IEEE, 23–26.
 - [33] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s verify step by step. In *The twelfth international conference on learning representations*.
 - [34] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. 2018. Reinforcement Learning on Web Interfaces using Workflow-Guided Exploration. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1802.08802>
 - [35] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. 2025. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326* (2025).
 - [36] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. 2024. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573* (2024).
 - [37] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillcrap. 2023. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems* 36 (2023), 59708–59728.
 - [38] Linxin Song, Yutong Dai, Viraj Prabhu, Jieyu Zhang, Taiwei Shi, Li Li, Junnan Li, Silvio Savarese, Zeyuan Chen, Jieyu Zhao, et al. 2025. Coact-1: Computer-using agents with coding as actions. *arXiv preprint arXiv:2508.03923* (2025).
 - [39] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, et al. 2024. Cradle: Empowering foundation agents towards general computer control. *arXiv preprint arXiv:2403.03186* (2024).
 - [40] Jianqiang Wan, Sibao Song, Wenwen Yu, Yuliang Liu, Wenqing Cheng, Fei Huang, Xiang Bai, Cong Yao, and Zhibo Yang. 2024. Omniparser: A unified framework for text spotting key information extraction and table recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 15641–15653.
 - [41] Bryan Wang, Gang Li, Xin Zhou, Zhouong Chen, Tovi Grossman, and Yang Li. 2021. Screen2words: Automatic mobile ui summarization with multimodal learning. In *The 34th Annual ACM Symposium on User Interface Software and Technology*, 498–510.
 - [42] Junyang Wang, Haiyang Xu, Haitao Jia, Xi Zhang, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration. *Advances in Neural Information Processing Systems* 37 (2024), 2686–2710.
 - [43] Xuehui Wang, Zhenyu Wu, Jingjing Xie, Zichen Ding, Bowen Yang, Zehao Li, Zhaoyang Liu, Qingyun Li, Xuan Dong, Zhe Chen, et al. 2025. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents. *arXiv preprint arXiv:2507.19478* (2025).
 - [44] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th annual international conference on Mobile computing and networking*, 543–557.
 - [45] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456* (2024).
 - [46] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. 2024. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. *arXiv preprint arXiv:2410.23218* (2024).
 - [47] Zhiheng Xi, Chenyang Liao, Guanyu Li, Yajie Yang, Wenxiang Chen, Zhihao Zhang, Binghai Wang, Senjie Jin, Yuhao Zhou, Jian Guan, et al. 2025. Agentprm: Process reward models for llm agents via step-wise promise and progress. *arXiv preprint arXiv:2511.08325* (2025).
 - [48] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Os-world: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems* 37 (2024), 52040–52094.
 - [49] Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, et al. 2026. Mobile-Agent-v3. 5: Multi-platform Fundamental GUI Agents. *arXiv preprint arXiv:2602.16855* (2026).
 - [50] Tianqi Xu, Linyao Chen, Dai-jie Wu, Yanjun Chen, Zecheng Zhang, Xiang Yao, Zhiqiang Xie, Yongchao Chen, Shilong Liu, Bochen Qian, Philip Torr, Bernard Ghanem, and Guohao Li. 2024. CRAB: Cross-environment Agent Benchmark for Multimodal Language Model Agents. [arXiv:2407.01511](https://arxiv.org/abs/2407.01511) [cs.AI] <https://arxiv.org/abs/2407.01511>
 - [51] Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianping Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, et al. 2026. Evocua: Evolving computer use agents via learning from scalable synthetic experience. *arXiv preprint arXiv:2601.15876* (2026).
 - [52] Haolong Yan, Yeqing Shen, Xin Huang, Jia Wang, Kaijun Tan, Zhixuan Liang, Hongxin Li, Zheng Ge, Osamu Yoshie, Si Li, et al. 2025. GUI Exploration Lab: Enhancing Screen Navigation in Agents via Multi-Turn Reinforcement Learning. *arXiv preprint arXiv:2512.02423* (2025).
 - [53] Haolong Yan, Jia Wang, Xin Huang, Yeqing Shen, Ziyang Meng, Zhimin Fan, Kaijun Tan, Jin Gao, Lieyu Shi, Mi Yang, et al. 2025. Step-gui technical report. *arXiv preprint arXiv:2512.15431* (2025).
 - [54] Zhen Yang, Zi-Yi Dou, Di Feng, Forrest Huang, Anh Nguyen, Keen You, Omar Attia, Yuhao Yang, Michael Feng, Haotian Zhang, et al. 2025. Ferret-ui lite: Lessons from building small on-device gui agents. *arXiv preprint arXiv:2509.26539* (2025).
 - [55] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. 2025. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144* (2025).
 - [56] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearingin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. 2024. Ferret-ui: Grounded mobile ui understanding with multimodal llms. In *European Conference on Computer Vision*. Springer, 240–255.
 - [57] Zhuosheng Zhan and Aston Zhang. 2023. You Only Look at Screens: Multimodal Chain-of-Action Agents. *arXiv preprint arXiv:2309.11436* (2023).
 - [58] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, et al. 2025. Ufo: A ui-focused agent for windows os interaction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 597–622.
 - [59] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. Appagent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 1–20.
 - [60] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanhao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
 - [61] Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. 2025. MAI-UI Technical Report: Real-World Centric Foundation GUI Agents. *arXiv preprint arXiv:2512.22047* (2025).
 - [62] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).
 - [63] Lianghui Zhu, Xinggang Wang, and Xinlong Wang. 2023. Judgelm: Fine-tuned large language models are scalable judges. *arXiv preprint arXiv:2310.17631* (2023).